

Reporte de la práctica

Temporización no bloqueante — Parte 2: tres LEDs independientes con millis()

1. Datos generales

Materia	Sistemas Programables
Institución	Instituto Tecnológico de Mazatlán (TecNM)
Alumno	Jesús Antonio Rodríguez Peraza — L231000105
Práctica	Práctica 2.1.1 (Parte 2) — Temporización no bloqueante: tres LEDs independientes con millis()
Placa utilizada	Arduino UNO R4 WiFi
Componentes	Arduino UNO R4 WiFi, 3 LEDs, 3 resistencias de 220 Ω , protoboard, cables Dupont
Programa	ParpadeoMillis.ino (Codigo/ParpadeoMillis/), incluye el reto opcional del serial cada 3 s
Repositorio	github.com/Jesus-Jazisenot/Universidad → Sistemas-Programables/Practica-2-Temporizacion-No-Bloqueante-Parte-2-millis

2. Objetivo

Hacer que tres LEDs parpadeen a 500, 1000 y 1500 ms al mismo tiempo y cada uno con su propio ritmo, sin un solo delay(), sustituyendo las esperas por comparaciones con millis(). Organizar las tareas en un struct y un arreglo para que agregar una tarea sea agregar un renglón, y comprobarlo con una cuarta tarea (reto opcional) que imprime en el Monitor Serie cada 3 s.

3. Diagrama del circuito (conexiones)

LED	Pin Arduino	Resistencia	Periodo pedido
LED 1	8	220 Ω a GND	500 ms
LED 2	9	220 Ω a GND	1000 ms
LED 3	10	220 Ω a GND	1500 ms

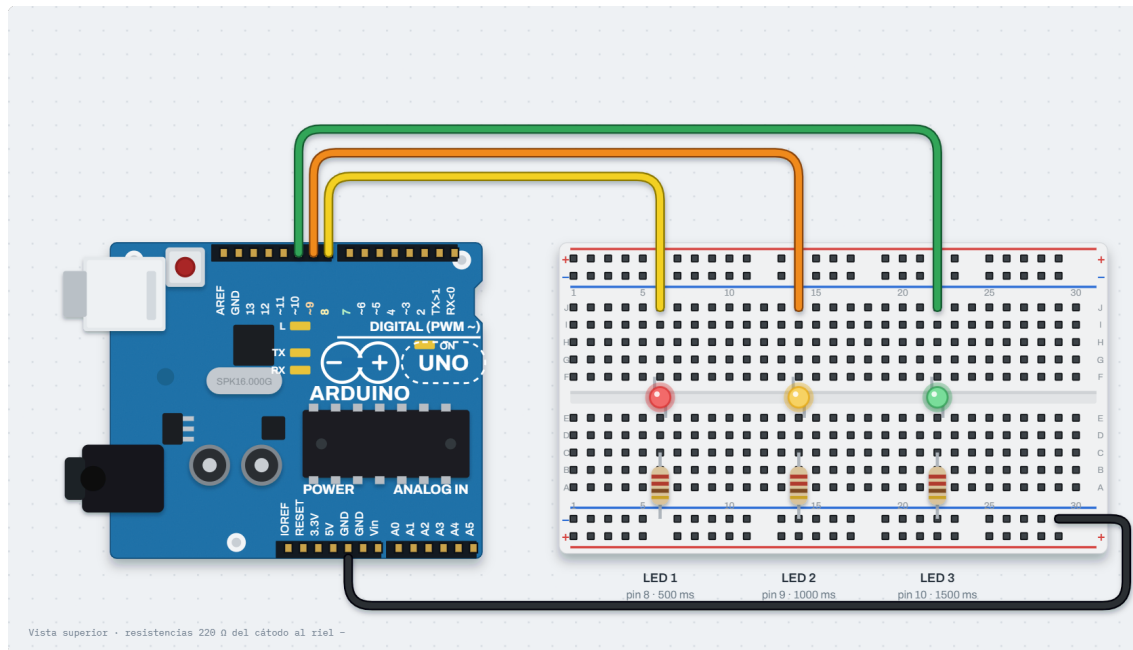


Figura 1. Diagrama de conexiones del circuito.

4. Procedimiento

1. Se usó exactamente el mismo circuito de la Parte 1: LEDs en los pines 8, 9 y 10 con resistencia de $220\ \Omega$ a GND.
2. Se cargó ParpadeoMillis.ino. Cada LED es una entrada del arreglo leds[] con su pin, su periodo, el instante de su último cambio y su estado.
3. En cada vuelta del loop() se lee millis() una sola vez y, por cada tarea, se evalúa if (ahora - ultimoCambio >= periodoMs); si se cumple, se invierte el LED y se anota el instante.
4. Se agregó la cuarta tarea del reto: un mensaje en el Monitor Serie (9600 baudios) cada 3000 ms con el número de mensaje, el tiempo y el estado de los tres LEDs.
5. Se observó el parpadeo simultáneo, se leyó el Monitor Serie y se grabó el video del funcionamiento.

5. Tabla de tiempos y comparación con la Parte 1

Estados de los tres LEDs durante los primeros 3 s. El patrón se repite cada 3000 ms, que es el mínimo común múltiplo de los tres periodos, y justo ahí cae el mensaje serial de la cuarta tarea:

Tiempo (ms)	LED1 (500)	LED2 (1000)	LED3 (1500)	LEDs encendidos
0	off	off	off	0
500	ON	off	off	1
1000	off	ON	off	1
1500	ON	ON	ON	3
2000	off	off	ON	1
2500	ON	off	ON	2
3000	off	ON	off	1

Comparación contra la Parte 1 en un mismo intervalo de 6 s:

LED	Periodo	Cambios de estado en 6 s	Encendidos en 6 s	Con delay() (Parte 1)
LED 1	500 ms	12	6	1
LED 2	1000 ms	6	3	1
LED 3	1500 ms	4	2	1

** La columna LEDs del Monitor Serie (por ejemplo 010) es el estado de LED1, LED2 y LED3 en el instante del mensaje: a los 3000 ms LED1 lleva 6 cambios (apagado), LED2 lleva 3 (encendido) y LED3 lleva 2 (apagado). La salida serial es transcripción de lo que imprime el programa.*

6. Salida del Monitor Serie (9600 baudios)

```
Parpadeo con millis(): 500 / 1000 / 1500 ms
Mensaje #1 t = 3000 ms LEDs: 010
Mensaje #2 t = 6000 ms LEDs: 000
Mensaje #3 t = 9000 ms LEDs: 010
Mensaje #4 t = 12000 ms LEDs: 000
Mensaje #5 t = 15000 ms LEDs: 010
Mensaje #6 t = 18000 ms LEDs: 000
```

7. Observaciones sobre el comportamiento del sistema

- Los tres LEDs parpadean al mismo tiempo, cada uno con su periodo; se observan momentos con dos LEDs encendidos (500 y 1500 ms coinciden cada 1500 ms) y a los 1500 ms los tres a la vez.
- El patrón completo se repite cada 3 s, y el mensaje serial cae exactamente en esos múltiplos (3000, 6000, 9000 ms), sin desfase acumulado.
- Agregar la cuarta tarea (serial) no alteró el ritmo de ningún LED: la temporización de cada tarea es independiente de las demás.
- Se organizó cada LED como una entrada de un struct Parpadeo en un arreglo; agregar un LED es agregar un renglón, no reescribir el loop().
- El mensaje del reto muestra el estado de los LEDs (por ejemplo 010) en el instante del envío, lo que sirve para verificar la tabla de tiempos sin cronómetro.

8. Conclusiones

La diferencia con la Parte 1 se ve a simple vista: ahora hay instantes con dos y hasta tres LEDs encendidos al mismo tiempo, y cada uno lleva su ritmo. Con el mismo circuito, lo único que cambió fue la forma de esperar.

El cambio es de mentalidad más que de código: en vez de "espera X ms" el programa dice "revisa si ya toca". Cada tarea recuerda cuándo actuó (ultimoCambio) y compara ahora - ultimoCambio >= periodo; como nadie detiene al procesador, el loop() gira miles de veces por segundo y todas las tareas se atienden.

La cuarta tarea del reto (mensaje serial cada 3 s) entró sin tocar una sola línea de los LEDs: solo se agregó otra comparación con su propio periodo. Eso demuestra que la estructura es realmente no bloqueante y que agregar tareas escala, a diferencia del enfoque con delay().

La resta en unsigned long resuelve el desbordamiento de millis() a los ~49.7 días: cuando el contador da la vuelta, la diferencia también da la vuelta y sigue siendo la diferencia real. Escribir ahora >=

ultimoCambio + periodo sí fallaría en ese momento.

El patrón tiene un límite: sigue siendo multitarea cooperativa. Si una tarea tarda mucho (un cálculo largo o un delay() escondido en una librería), retrasa a todas las demás. Para garantías de tiempo estrictas se necesitan interrupciones o temporizadores por hardware.

9. Evidencia del armado

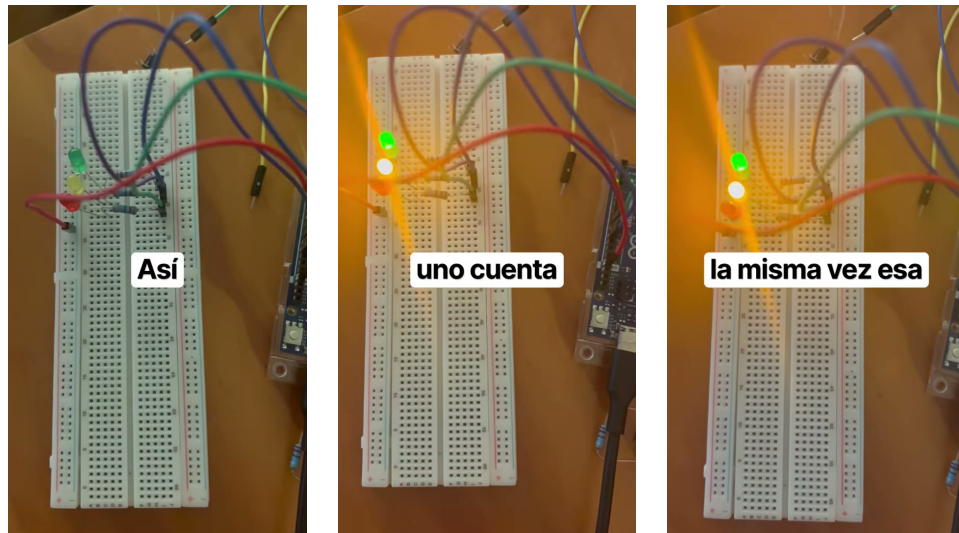


Figura 2. Circuito armado; a diferencia de la Parte 1 se ven dos LEDs encendidos a la vez (cuadros del video).

Video del funcionamiento: https://www.youtube.com/shorts/pJ0OG_M3bCs